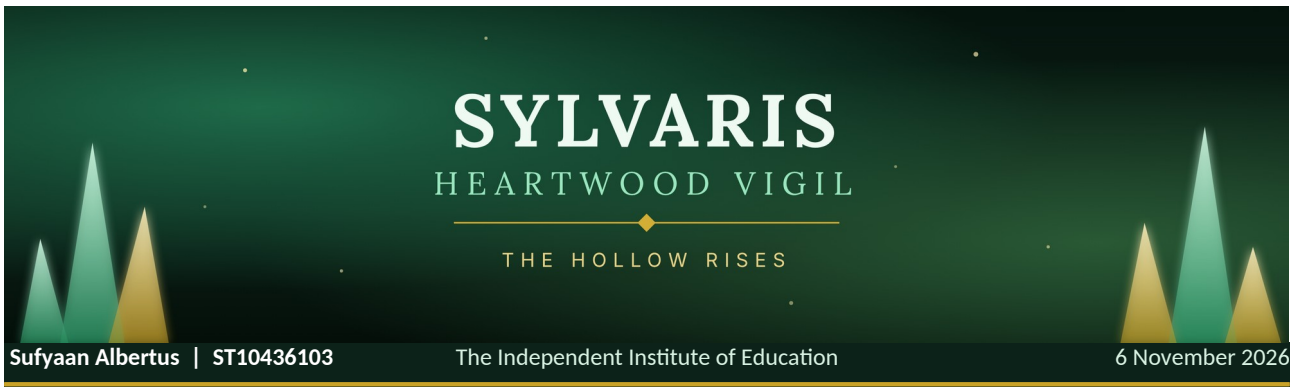


Sufyaan Albertus | ST10436103 The Independent Institute of Education 6 November 2026

Void Heralds: Planning a Procedurally Generated Mini-Boss

GADE7322 PORTFOLIO OF EVIDENCE - PART 3 - PLANNING DOCUMENT (INDIVIDUAL)

1. Introduction

This document plans the custom procedural generation feature for Sylvaris: Heartwood Vigil, titled Void Heralds. Procedural content generation is the algorithmic creation of game content with limited human input (Shaker, Togelius and Nelson, 2016). A Void Herald is a mini-boss generated at runtime rather than designed by hand: its body, traits, statistics, colours, size and name are all derived from a single seed. The feature addresses a weakness identified in Part 2, where waves became harder but lacked memorable variation. Heralds create distinct encounters that require the player to read the threat and adapt their strategy.

2. Research and Rationale

Elite monsters in Diablo III receive randomly selected affixes, and the number of affixes increases with difficulty (Diablo Wiki, n.d.). Heralds adopt the same principle, gaining traits as the waves progress.

Valve's AI Director in Left 4 Dead makes pacing decisions at runtime, but only within constraints chosen by the designers, which keeps encounters unpredictable yet fair (Booth, 2009). Sylvaris already has an adaptive Wave Director, so trait selection is weighted against the player's detected play style. Seeded pseudo-random and grammar-based generation (Shaker, Togelius and Nelson, 2016) inform the reproducible seeding and the syllable-based names.

3. Generation Pipeline

The generator is constructive, building content in a single pass without a later search (Togelius et al., 2011), so it is fast enough to run during play. Each Herald is produced in five stages (Figure 1) from a seed that combines the run seed with the wave number. This guarantees variety between runs while allowing any Herald to be reproduced. Table 1 lists the rules.

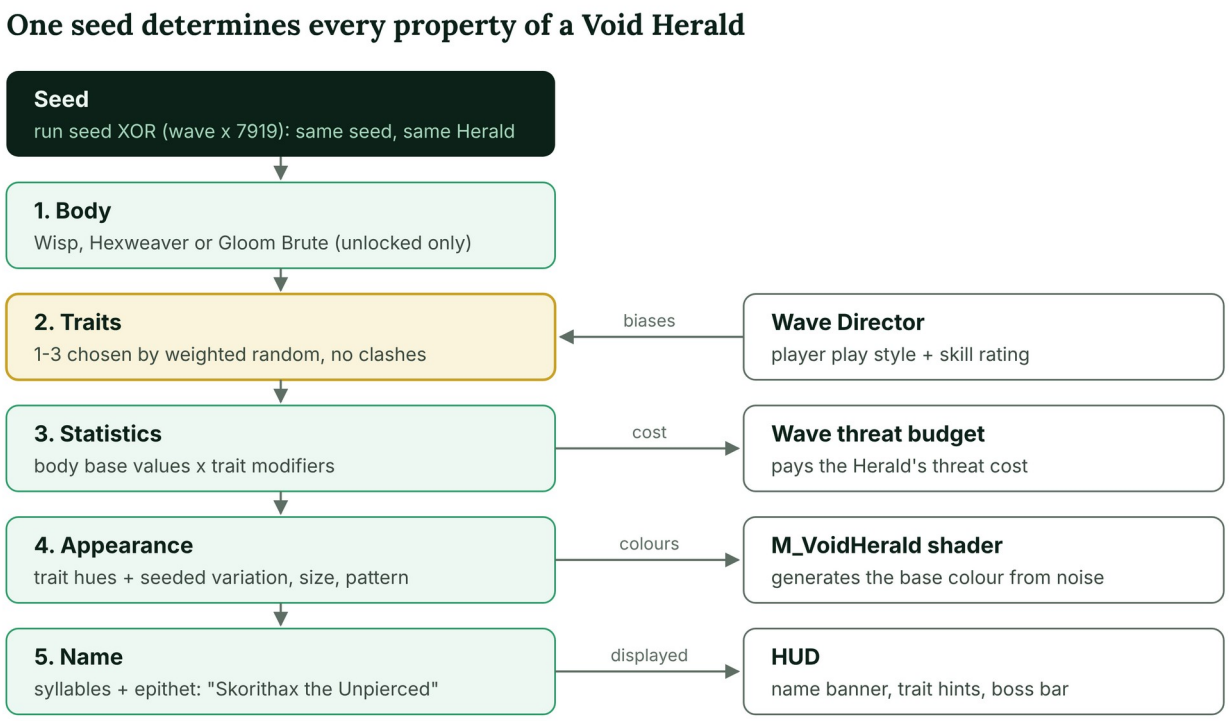


Figure 1: The Void Herald generation pipeline and the systems it connects to.

Table 1: Generation and scaling rules.

Rule	Value
Frequency	Every third wave, starting on wave 3
Seed	Run seed XOR (wave x 7919), so the same run and wave always give the same Herald
Body	Weighted pick from unlocked types: Wisp 1, Hexweaver 1.2, Gloom Brute 1.5
Trait count	1 trait; 2 from wave 7; 3 from wave 12; +1 if skill rating is 1.25 or higher (maximum 3)
Base statistics	Wisp: 9x health (+35% per wave), 1.9x size. Hexweaver: 4x, 1.5x. Brute: 3.2x, 1.35x. All: 0.85x speed
Threat cost	6 + 3 per trait (+2 for Colossal), paid from the wave budget
Reward	3x Starlight + 1x per trait
Lane	Builder: their strongest lane (to test it). Other styles: their weakest lane

4. Trait Design and Adaptive Weighting

Adams (2014) argues that a challenge only feels fair when the player can perceive it, so each trait changes both the Herald's behaviour and its appearance (Table 2). Traits are drawn by weighted random selection without replacement, and clashing traits such as Colossal and Swift are never combined. The weights shift with play style: an arrow-focused Hunter is three times as likely to face a Warded Herald, while a tower-focused Builder mostly meets Siegebreakers (Figure 2).

Table 2: Herald traits and the play style each is weighted against.

Trait	Effect	Weighted against
Colossal	Double health, 35% larger, slower	-
Swift	60% faster, reduced health	Hunter (arrow-focused)

Trait	Effect	Weighted against
Warded	Arrows deal half damage	Hunter
Regenerating	Heals 3% per second when not hit	Hunter
Siegebreaker	Damages nearby towers	Builder (tower-focused)
Rootbane	Immune to Warden slows	Builder
Brood	Splits into three to six Wisps on death	Builder

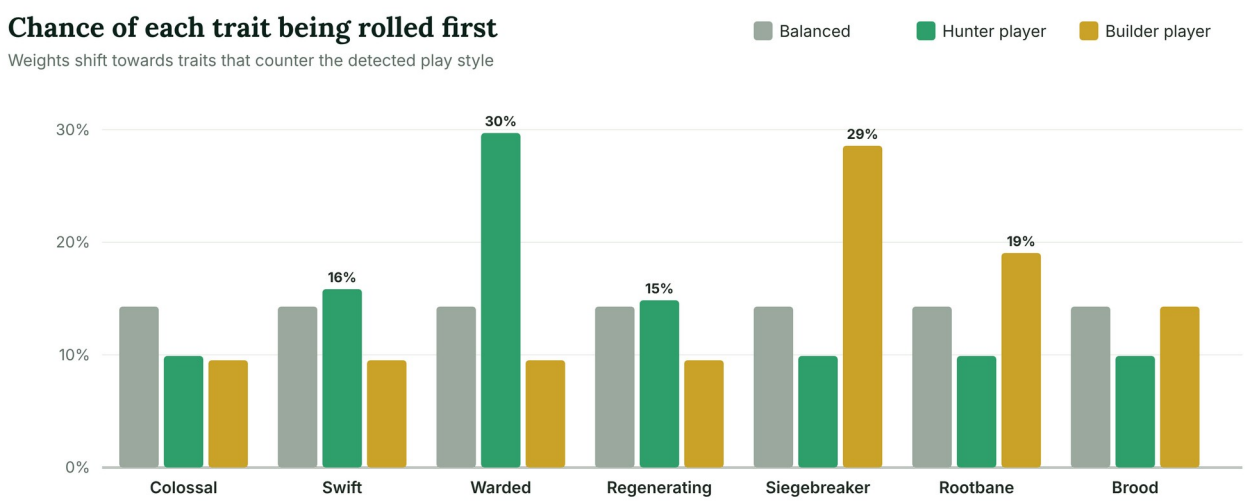


Figure 2: First-trait probabilities calculated from the planned trait weights for each play style.

5. Appearance and Naming

Each trait owns a signature hue. The first trait sets the primary colour, the second the secondary colour and the last the glow, with seeded variation. These values drive the custom M_VoidHerald shader, which builds the base colour from gradient noise (Perlin, 1985). Names join two or three syllables to an epithet from the main trait, such as Skorithax the Unpierced (Figure 3).

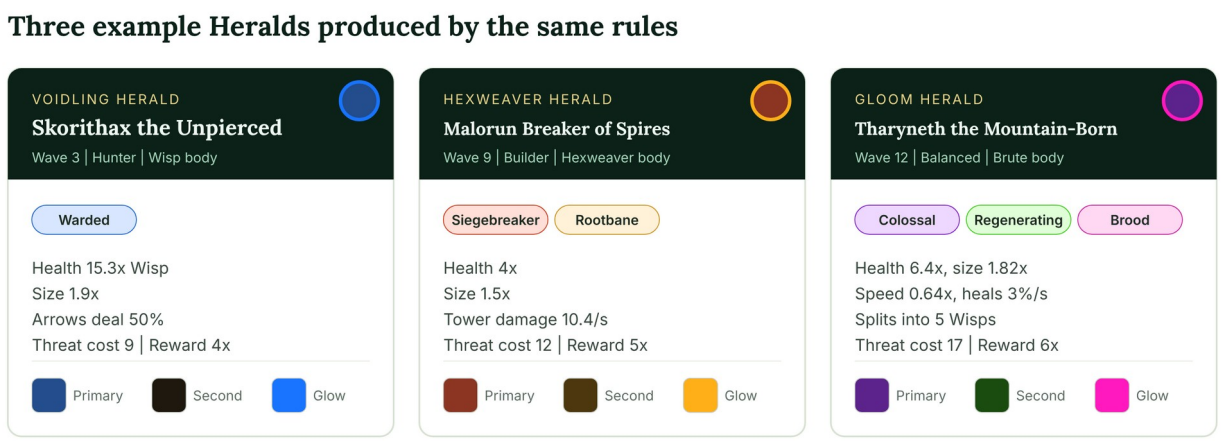


Figure 3: Illustrative Heralds generated from the rules above for three different contexts.

6. Integration and Balance

The Herald is part of the wave rather than a separate event: the Wave Director pays its threat cost from the wave budget and spawns it last. The interface shows a name banner, trait hints, a boss bar and a screen-edge

corruption effect, and Table 3 lists the classes involved. Following Smith and Whitehead (2010), traits will be balanced across many generated outputs, with a fixed debug seed used to reproduce any case.

Table 3: Planned implementation in Unreal Engine 5 (C++).

Class or asset	Responsibility
`HeraldGenerator`	Generate(context) returns a complete Herald profile from the seed
`FHeraldProfile`	Struct holding body, traits, multipliers, colours and name
`UWaveDirectorComponent`	Decides Herald waves, pays the threat cost, chooses the lane
`ASylvarisEnemyBase`	ApplyHeraldProfile() applies stats, traits and shader colours
`M_VoidHerald`	Custom shader: noise base colour, glowing veins, spikes
`ASylvarisHUD`	Name banner, trait hints and boss health bar
`ASylvarisScreenEffects`	Purple corruption at the screen edges while a Herald lives

7. Conclusion

Void Heralds combine weighted randomness, adaptive constraints and grammar-based naming so that every run produces different encounters. Lecturer feedback will be obtained before implementation.

References

Adams, E. 2014. *Fundamentals of game design*. 3rd ed. Berkeley: New Riders.

Booth, M. 2009. *Replayable cooperative game design: Left 4 Dead*. Game Developers Conference, San Francisco, 23-27 March 2009. [Online]. Available at: https://cdn.fastly.steamstatic.com/apps/valve/2009/GDC2009_ReplayableCooperativeGameDesign_Left4Dead.pdf [Accessed 6 November 2026].

Diablo Wiki. n.d. *Elite*. [Online]. Available at: https://di.diablowiki.net/Elite [Accessed 6 November 2026].

Perlin, K. 1985. An image synthesizer. *ACM SIGGRAPH Computer Graphics*, 19(3): 287-296.

Shaker, N., Togelius, J. and Nelson, M.J. 2016. *Procedural content generation in games*. Cham: Springer.

Smith, G. and Whitehead, J. 2010. Analyzing the expressive range of a level generator. In: *Proceedings of the 2010 Workshop on Procedural Content Generation in Games*. Monterey, 18 June 2010. New York: ACM: 1-7.

Togelius, J., Yannakakis, G.N., Stanley, K.O. and Browne, C. 2011. Search-based procedural content generation: a taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3): 172-186.